Day 105 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 16, Day 1 reference

Watch: Decision and Classification Trees, Clearly Explained!!! (StatQuest) | Gini & Entropy versus misclassification error (Sebastian Raschka)

`day105_decision_trees.py`, segment by segment, in the order `main()` uses things. Everything is NumPy; scikit-learn supplies only the bundled dataset, and Day 99's bootstrap is imported. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: p_k = share of class k in a group · I = impurity (Gini or entropy) · L, R = the two sides of a split · f, t = feature and threshold

Segment 0 — Setup

Imports and settings

CODE

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.join(HERE, "..", "week-15-practical-nlp-pretrained"))
with contextlib.redirect_stdout(io.StringIO()):
    import day99_exercise as d99
TITANIC = os.path.join(HERE, "..", "week-03-intro-to-machine-learning", "titanic.csv")
DEPTHS = list(range(1, 11)) + [None]
MIN_LEAF = 1
```

- `from sklearn.datasets import load_breast_cancer` — only the data loader; no scikit-learn model is used.
- `DEPTHS = list(range(1, 11)) + [None]` — list concatenation: depths 1 to 10, then None for 'no limit'.

Segment 1 — Impurity and the split

gini and entropy, for many groups at once

MATH

$$Gini = 1 - \Sigma\, p_k^{\,2}, \quad H = -\Sigma\, p_k \log_2 p_k$$

CODE

```
def gini(counts):
    n = counts.sum(-1, keepdims=True)
    p = counts / np.maximum(n, 1)
    return 1 - (p ** 2).sum(-1)
def entropy(counts):
    n = counts.sum(-1, keepdims=True)
    p = counts / np.maximum(n, 1)
    with np.errstate(divide="ignore", invalid="ignore"):
        return np.where(p > 0, -p * np.log2(p), 0.0).sum(-1)
```

- `counts.sum(-1, keepdims=True)` — sums over the last axis but keeps it (size 1), so the division broadcasts row by row.
- `np.maximum(n, 1)` — avoids dividing by zero for an empty group.
- `np.errstate(divide="ignore", invalid="ignore")` — silences the warning for log2(0); np.where then replaces those terms with 0.

best_split: every threshold of a feature at once

MATH

$$gain(t) = I(S) - (n_L/n)\, I(L) - (n_R/n)\, I(R), \text{ for every gap } t \text{ at once}$$

CODE

```
def best_split(X, y, n_classes, impurity, min_leaf=1):
    n, best = len(y), None
    parent = impurity(np.bincount(y, minlength=n_classes)[None])[0]
    for f in range(X.shape[1]):
        order = np.argsort(X[:, f], kind="stable"); xs, ys = X[order, f], y[order]
        left = np.cumsum(np.eye(n_classes, dtype=int)[ys], axis=0)[:-1]
        right = np.bincount(y, minlength=n_classes) - left
        nl = np.arange(1, n)
        ok = (xs[1:] != xs[:-1]) & (nl >= min_leaf) & (n - nl >= min_leaf)
        if not ok.any():
            continue
        child = (nl * impurity(left) + (n - nl) * impurity(right)) / n
        gain = np.where(ok, parent - child, -np.inf)
        i = int(np.argmax(gain))
        if best is None or gain[i] > best[2] + 1e-12:
            best = (f, (xs[i] + xs[i + 1]) / 2, float(gain[i]))
    return best if best and best[2] > 1e-12 else None
```

- `np.argsort(X[:, f], kind="stable")` — the order that sorts this feature; 'stable' keeps ties in their original order.
- `np.eye(n_classes, dtype=int)[ys]` — one-hot rows: indexing the identity matrix with the labels.
- `np.cumsum(..., axis=0)[:-1]` — running class counts: row i is the counts of the first i + 1 sorted examples.
- `(xs[1:] != xs[:-1])` — a gap between two different values; equal neighbours cannot be separated.

Segment 2 — The tree

grow: recursion into a nested dictionary

CODE

```
def grow(X, y, n_classes, impurity, depth=0, max_depth=None, min_leaf=MIN_LEAF):
    counts = np.bincount(y, minlength=n_classes)
    node = {"counts": counts, "pred": int(np.argmax(counts))}
    if (max_depth is not None and depth >= max_depth) or counts.max() == len(y):
        return node
    s = best_split(X, y, n_classes, impurity, min_leaf)
    if s is None:
        return node
    f, t, g = s; m = X[:, f] <= t
    node.update(feature=f, threshold=t, gain=g,
                left=grow(X[m], y[m], n_classes, impurity, depth + 1, max_depth, min_leaf),
                right=grow(X[~m], y[~m], n_classes, impurity, depth + 1, max_depth, min_leaf))
    return node
```

- `node = {"counts": counts, "pred": ...}` — every node is a dictionary; a leaf has no 'feature' key.
- `f, t, g = s; m = X[:, f] <= t` — tuple unpacking, then a boolean mask of the rows that answer yes.
- `X[~m]` — ~ inverts a boolean array: the rows that answer no.
- `left=grow(...), right=grow(...)` — the function calls itself on each side: recursion, one level deeper each time.

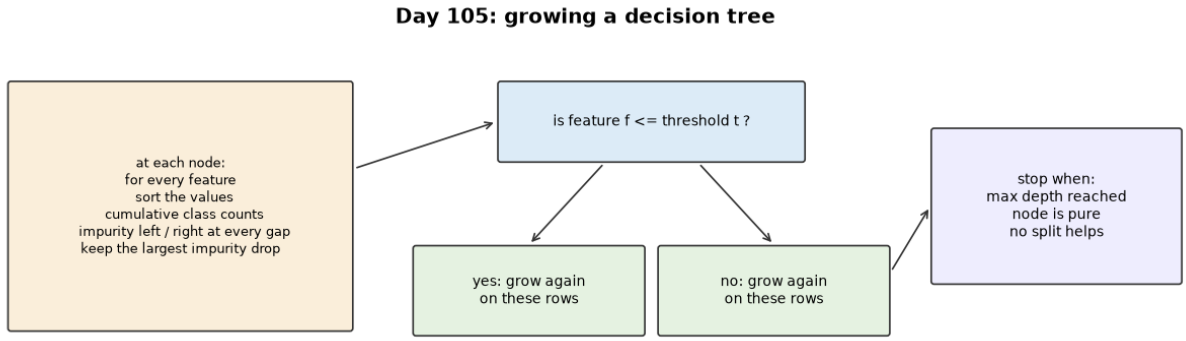
predict, size and show

CODE

```
def predict(node, X):
    out = np.empty(len(X), dtype=int)
    for i, x in enumerate(X):
        n = node
        while "feature" in n:
            n = n["left"] if x[n["feature"]] <= n["threshold"] else n["right"]
        out[i] = n["pred"]
    return out
def size(node):
    if "feature" not in node:
        return 1, 0
    (a, da), (b, db) = size(node["left"]), size(node["right"])
    return a + b, 1 + max(da, db)
def show(node, names, classes, indent="  "):
    if "feature" not in node:
        return [f"{indent}-> {classes[node['pred']]}  {node['counts'].tolist()}"]
    q = f"{names[node['feature']]} <= {node['threshold']:.3g}"
    return ([f"{indent}if {q}:"] + show(node["left"], names, classes, indent + "    ") +
            [f"{indent}else:"] + show(node["right"], names, classes, indent + "    "))
```

- `while "feature" in n:` — walk down until a node without a question: a leaf.
- `(a, da), (b, db) = size(...), size(...)` — nested tuple unpacking of two recursive results.
- `[...] + show(...) + [...]` — list concatenation builds the printed rules, indented one level per depth.

SYSTEM DIAGRAM



Growing a decision tree.

Segment 3 — Evaluation

stratified_split and kfold_accuracy

CODE

```
def stratified_split(y, test_frac, seed):
    rng = np.random.RandomState(seed); tr, te = [], []
    for c in np.unique(y):
        idx = rng.permutation(np.where(y == c)[0]); k = int(round(len(idx) * test_frac))
        te += list(idx[:k]); tr += list(idx[k:])
    return np.array(sorted(tr)), np.array(sorted(te))
def kfold_accuracy(X, y, impurity, max_depth, k=5, seed=0):
    folds = np.array_split(np.random.RandomState(seed).permutation(len(y)), k)
    accs = []
    for i in range(k):
        va = folds[i]; tr = np.concatenate([folds[j] for j in range(k) if j != i])
        t = grow(X[tr], y[tr], 2, impurity, max_depth=max_depth)
        accs.append(np.mean(predict(t, X[va]) == y[va]))
    return np.mean(accs), np.std(accs)
```

- `rng.permutation(np.where(y == c)[0])` — the indices of one class, shuffled; splitting each class keeps the balance.
- `np.array_split(..., k)` — k nearly equal folds, even when the length is not divisible by k.
- `np.concatenate([folds[j] for j in range(k) if j != i])` — every fold except the i-th: the training part of this round.

Segment 4 — The experiments

demo_instability: bootstrap trees and their vote

CODE

```
for b in range(30):
    s = rng.randint(0, len(tr), len(tr)); idx = tr[s]
    t = grow(X[idx], y[idx], 2, gini, max_depth=best)
    roots[names[t["feature"]]] += 1; p = predict(t, X[te]); P.append(p); accs.append(np.mean(p ==
        y[te]))
P = np.array(P); vote = (P.mean(0) > 0.5).astype(int)
```

- `rng.randint(0, len(tr), len(tr))` — len(tr) random positions WITH replacement: a bootstrap resample.

- `roots[names[t["feature"]]] += 1` — a Counter tallies the root question of each tree.
- `(P.mean(0) > 0.5).astype(int)` — the majority vote: the share of trees saying 1, thresholded at a half.

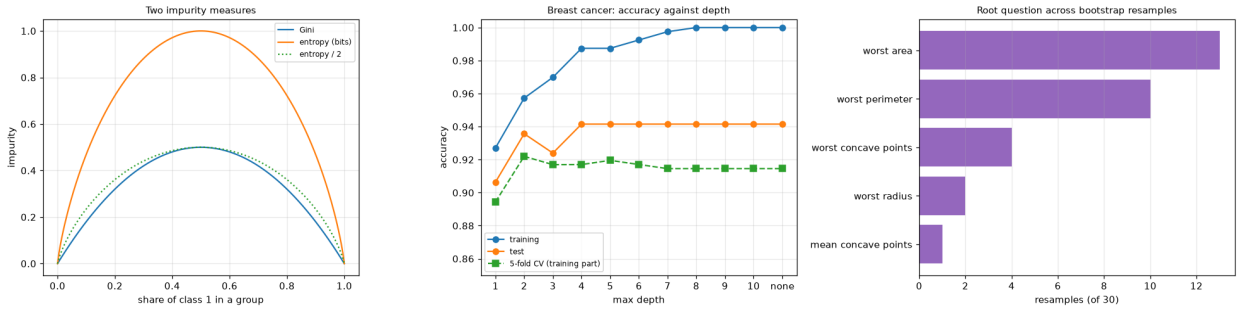
demo_titanic: a readable tree

CODE

```
d = pd.read_csv(TITANIC)
feats = ["Pclass", "is_female", "Age", "SibSp", "Parch", "Fare"]
d["is_female"] = (d["Sex"] == "female").astype(float)
d["Age"] = d["Age"].fillna(d["Age"].median())
X, y = d[feats].to_numpy(float), d["Survived"].to_numpy()
tr, te = stratified_split(y, 0.3, 0)
t = grow(X[tr], y[tr], 2, gini, max_depth=3)
```

- `(d["Sex"] == "female").astype(float)` — a text column turned into a 0/1 feature.
- `d["Age"].fillna(d["Age"].median())` — missing ages replaced by the median: simple, and a choice the tree then trusts.
- `d[feats].to_numpy(float)` — a pandas table to a NumPy array of floats.

REAL GRAPH



Real graph from the script: impurity curves, accuracy against depth, and the root question across 30 bootstrap resamples.

Quick Syntax Glossary — new or reinforced constructs

- `np.eye(k)[labels]` — one-hot encoding by indexing the identity matrix.
- `np.cumsum(a, axis=0)` — running totals down the rows.
- `np.argsort(a, kind="stable")` — sorting order that keeps equal values in their original order.
- `np.array_split(a, k)` — k nearly equal pieces.
- `~mask` — element-wise NOT of a boolean array.
- `def f(...): ... f(...)` — recursion: a function that calls itself on a smaller problem.

End of Day 105 syntax reference. For the experiments and what they showed, see the Day 105 complete guide.